Authentication and Authorization

Authentication

Authentication is the process by which people prove they are who they say they are. It's composed of two parts: a public statement of identity (usually in the form of a *username*) combined with a private response to a challenge (such as a *password*). The secret response to the authentication challenge can be based on one or more factors—something you know (a secret word, number, or passphrase for example), something you have (such as a smartcard, ID tag, or code generator), or something you are (like a biometric factor like a fingerprint or retinal print). A password by itself, which is a means of identifying yourself through something only you should know (and today's most common form of challenge response), is an example of *single-factor authentication*. This is not considered to be a strong authentication method, because a password can be intercepted or stolen in a variety of ways—for example, passwords are frequently written down or shared with others, they can be captured from the system or the network, and they are often weak and easy to guess.

Imagine if you could only identify your friends by being handed a previously agreed secret phrase on a piece of paper instead of by looking at them or hearing their voice. How reliable would that be? This type of identification is often portrayed in spy movies, where a secret agent uses a password to impersonate someone the victim is supposed to meet but has never seen. This trick works precisely because it is so fallible—the password is the only means of identifying the individual. Passwords are just not a good way of authenticating someone.

Unfortunately, password-based authentication was the easiest type to implement in the early days of computing, and the model has persisted to this day.

Other single-factor authentication methods are better than passwords. Tokens and smart cards are better than passwords because they must be in the physical possession of the user. Biometrics, which use a sensor or scanner to identify unique features of individual body parts, are better than passwords because they can't be shared—the user must be present to log in. However, there are ways to defeat these methods. Tokens and cards can be lost or stolen, and biometrics can be spoofed. Yet, it's much more difficult to do that than to steal or obtain a password. Passwords are the worst possible method of proving an identity, despite being the most common method.

Multifactor authentication refers to using two or more methods of checking identity. These methods include (listed in increasing order of strength):

- Something you know (a password or PIN code)
- Something you have (such as a card or token)
- Something you are (a unique physical characteristic)

Two-factor authentication is the most common form of multifactor authentication, such as a password-generating token device with an LCD screen that displays a number (either time based or sequential) along with a password, or a smart card along with a password. Again, passwords aren't very good choices for a second factor, but they are ingrained into our technology and collective consciousness, they are built into all computer systems, and they are convenient and cheap to implement. A token or smart card along with biometrics would be much better—this combination is practically impossible to defeat. However, most organizations aren't equipped with biometric devices.

The following sections provide a more detailed introduction to these types of authentication systems available today:

- Systems that use username and password combinations, including Kerberos
- Systems that use certificates or tokens
- Biometrics

Usernames and Passwords

In the familiar method of password authentication, a challenge is issued by a computer, and the party wishing to be identified provides a response. If the response can be validated, the user is said to be authenticated, and the user is allowed to access the system. Otherwise, the user is prevented from accessing the system.

Other password-based systems, including Kerberos, are more complex, but they all rely on a simple fallacy: they trust that anyone who knows a particular user's password is that user. Many password authentication systems exist. The following types of systems are commonly used today:

- Local storage and comparison
- Central storage and comparison

- Challenge and response
- Kerberos
- One-time password (OTP)

Each type of system is discussed in turn next.

One-Time Password Systems

Two problems plague passwords. First, they are (in most cases) created by people. Thus, people need to be taught how to construct strong passwords, and most people aren't taught (or don't care enough to follow what they're taught). These strong passwords must also be remembered and not written down, which means, in most cases, that long passwords cannot be required. Second, passwords do become known by people other than the individual they belong to. People do write passwords down and often leave them where others can find them. People commonly share passwords despite all your warnings and threats.

Passwords are subject to a number of different attacks. They can be captured and cracked, or used in a replay attack in which the passwords are intercepted and later used to repeat authentication.

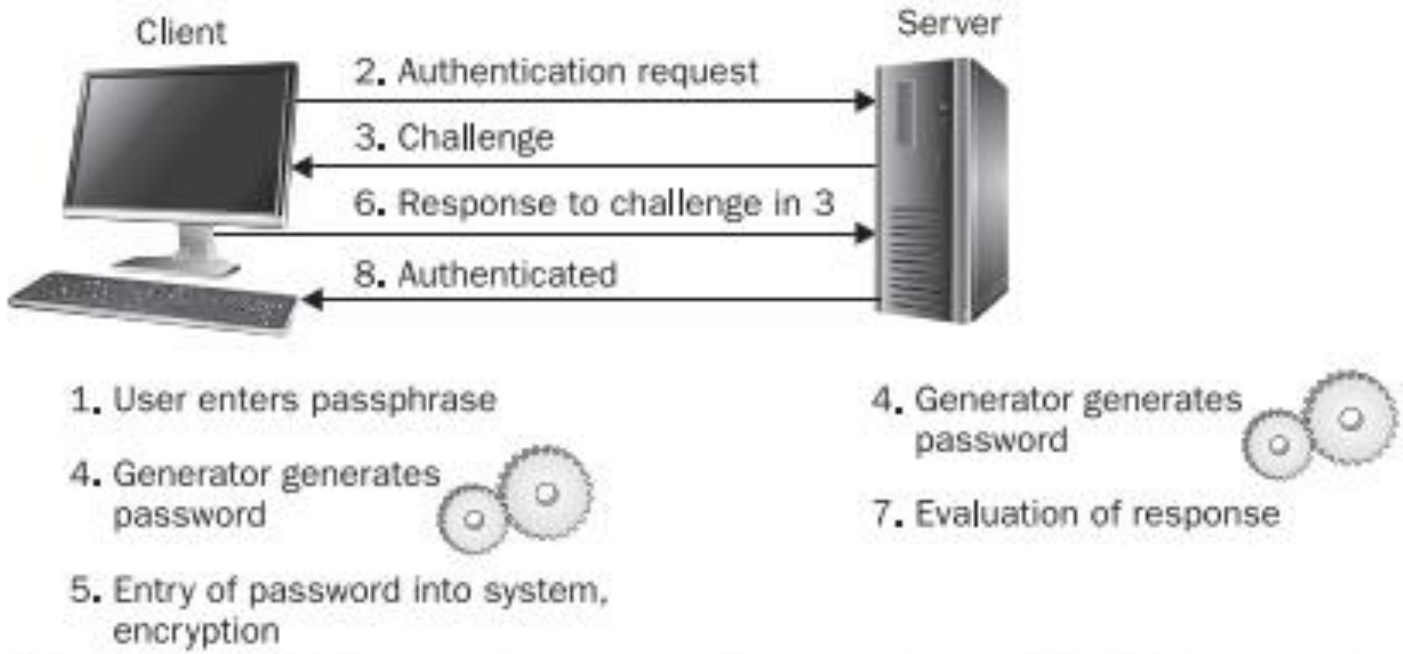


Figure 7-2 The S/Key one-time password process is a modified challenge and response authentication system.

4. The generator on the client and the generator on the server generate the same one-time password.
5. The generated password is displayed to the user for entry or is directly entered by the system. The password is used to encrypt the response.
6. The response is sent to the server.
7. The server creates its own encryption of the challenge using its own generated password, which is the same as the client's. The response is evaluated.
8. If there is a match, the user is authenticated.

Encryption

Keeping secrets by disguising them, hiding them, or making them indecipherable to others is an ancient practice. It evolved into the modern practice of cryptography—the science of secret writing, or the study of obscuring data using algorithms and secret keys.

A Brief History of Encryption

Once upon a time, keeping data secret was not hard. Hundreds of years ago, when few people were literate, the use of written language alone often sufficed to keep information from becoming general knowledge. To keep secrets then, you simply had to write them down, keep them hidden from those few people who could read, and prevent others from learning how to read. Deciphering the meaning of a document is difficult if it is written in a language you do not know.

History tells us that important secrets were kept by writing them down and hiding them from literate people. Persian border guards in the fourth century B.C. let blank wax writing tablets pass, but the tablets hid a message warning Greece of an impending attack. The message was simply covered by a thin layer of fresh wax. Scribes also tattooed messages on the shaved heads of messengers. When their hair grew back in, the messengers could travel incognito through enemy lands. When they arrived at their destination, their heads were shaved and the knowledge was revealed. But the fate of nations could not rely for very long on such obfuscation, which relied entirely on these tricks to keep the writing secret. It did not take very long for ancient military leaders to create and use more sophisticated techniques. Ever since then, encryption has been a process of using increasingly complex tactics to stay one step ahead of those who want your secret data.

Symmetric Key Cryptography

Symmetrical Key Cryptography also known as conventional or single-key encryption was the primary method of encryption before the introduction of public key cryptography in the 1970s. In symmetric-key algorithms, the same keys are used for data encryption and decryption. This type of cryptography plays a crucial role in securing data because the same key is used for both encryption and decryption.

In this article, we will cover the techniques used in symmetric key cryptography, its applications, principles on which it works, its types and limitations as well as what type of attacks in the digital world it gets to face.

Techniques Used in Symmetric Key Cryptography

Substitution and **Transposition** are two principal techniques used in symmetric-key cryptography.

Substitution Techniques

The symmetric key cryptographic method employs one secret key for the operations of encryption and decryption. Substitution techniques provide two significant approaches, wherein elements (letters, characters) from the plaintext message are replaced with new elements according to the rules based on the secret key.

- **Caesar Cipher:** Caesar cipher has since their predictability is so complete and no complexity is invested.

- **Monoalphabetic Ciphers:** This is where the ciphers use one rule of substitution throughout the message. This may involve replacing letters with numbers, symbols, or another set of letters in another order.

- **Playfair Cipher:** Implementation of repeated letters or letter pairs can expose patterns, and cryptanalysis techniques exist to exploit them.

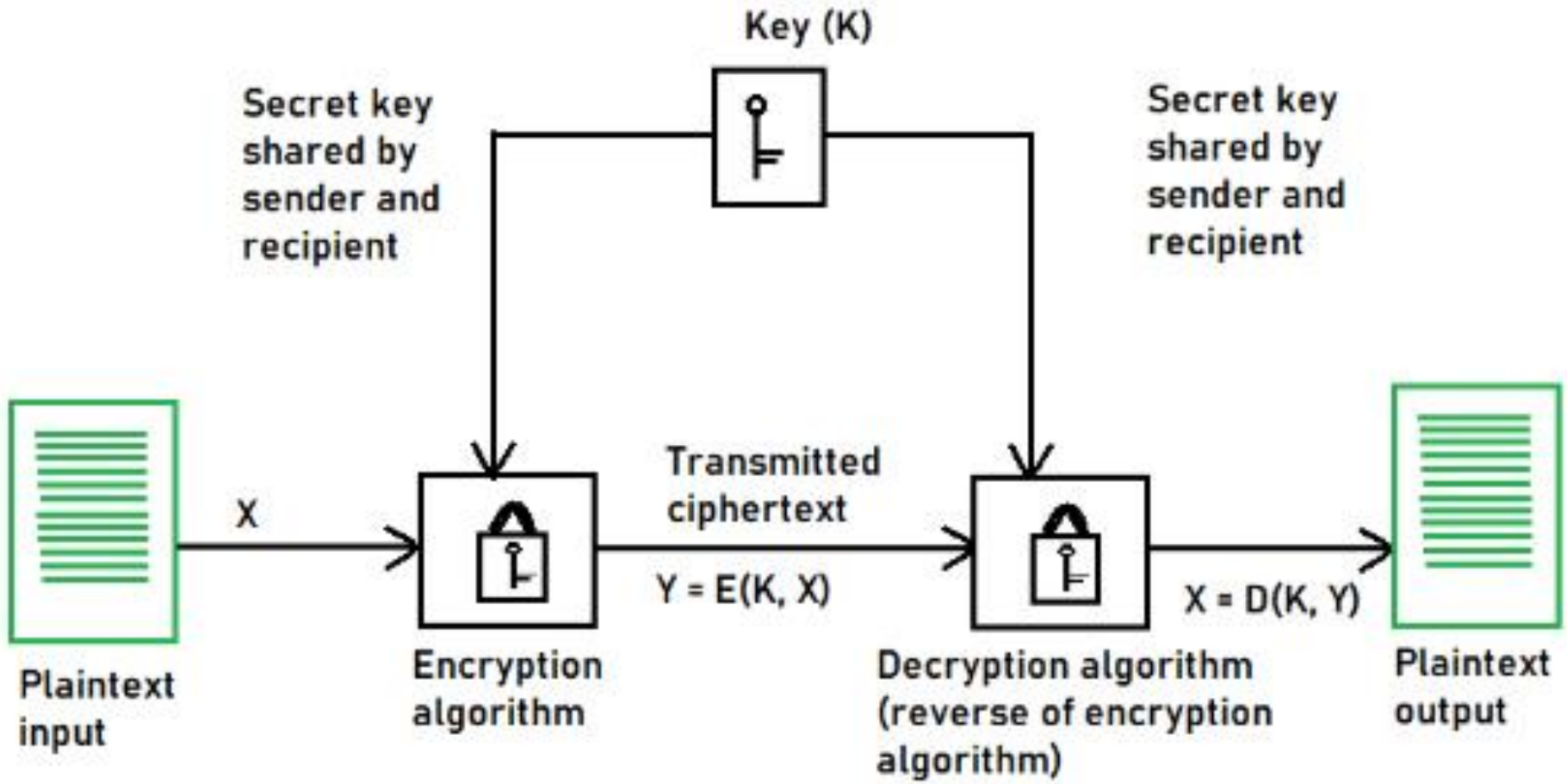


Diagram of Symmetric Encryption

Types of Symmetric Key Cryptography

1. Stream Ciphers
2. Block Ciphers

Stream Ciphers

The encryption process begins with the stream cipher's algorithm generating a pseudo-random keystream made up of the encryption key and the unique randomly generated number known as the nonce. The result is a random stream of bits corresponding to the length of the ordinary plaintext. Then, the ordinary plaintext is also deciphered into single bits.

These bits are then joined one by one to the keystream bits, gradually converting the ordinary plaintext into the ciphertext using the XOR bitwise operations. When the recipient wants to decrypt the encrypted plaintext, they must generate a new keystream made during the encryption. The encrypted plaintext is then deciphered one by one to derive the encrypted plaintext at the recipient's end.

Block Cipher

The result of a block cipher is a sequence of blocks that are then encrypted with the key. The output is a sequence of blocks of encrypted data in a specific order. When the ciphertext travels to its endpoint, the receiver uses the same cryptographic key to decrypt the ciphertext blockchain to the plaintext message.

Public Key Encryption

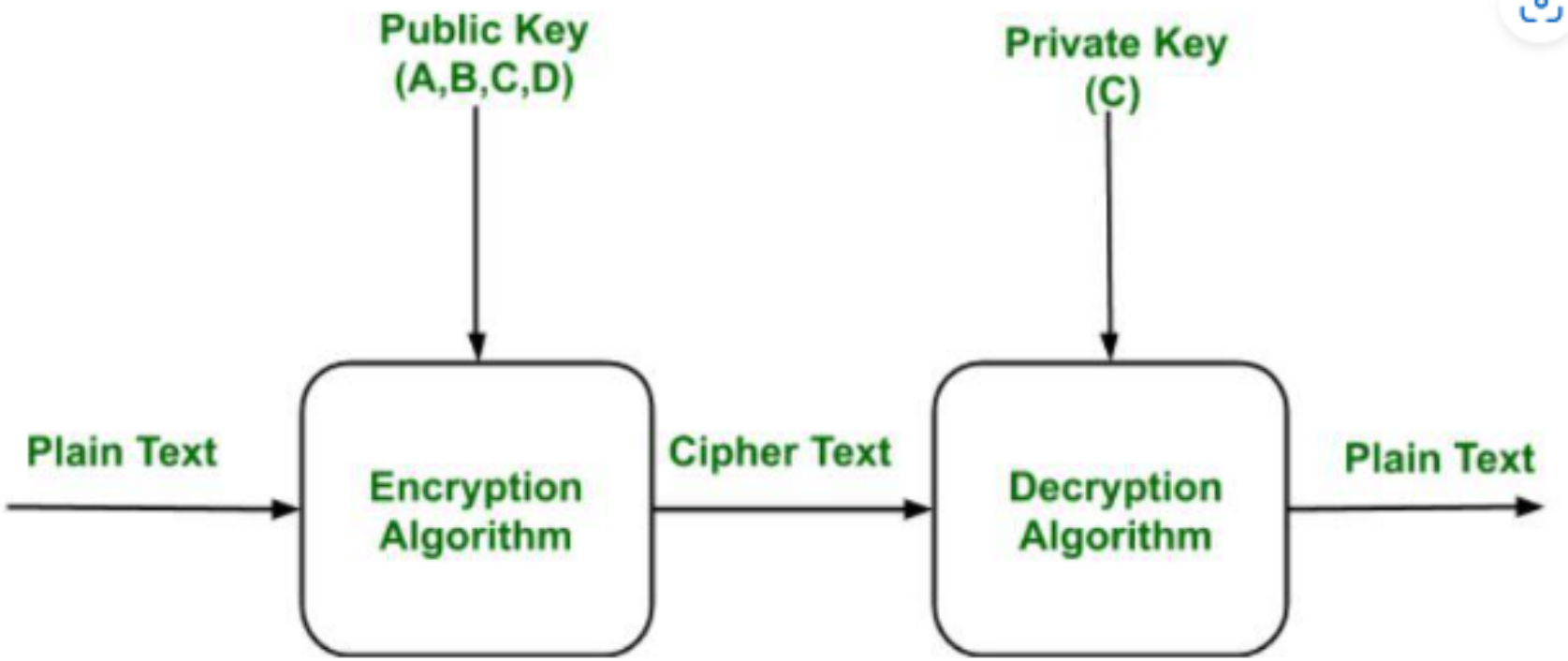
Public key cryptography provides a secure way to exchange information and authenticate users by using pairs of keys. The public key is used for encryption and signature verification, while the private key is used for decryption and signing. When the two parties communicate with each other to transfer the intelligible or sensible message, referred to as plaintext, is converted into apparently random unreadable for security purposes referred to as **ciphertext**.

What is Public Key Cryptography?

Public key cryptography is a method of secure communication that uses a pair of keys, a public key, which anyone can use to encrypt messages or verify signatures, and a private key, which is kept secret and used to decrypt messages or sign documents. This system ensures that only the intended recipient can read an encrypted message and that a signed message truly comes from the claimed sender. Public key cryptography is essential for secure internet communications, allowing for confidential messaging, authentication of identities, and verification of data integrity.

Example:

Public keys of every user are present in the Public key Register. If B wants to send a confidential message to C, then B encrypt the message using C Public key. When C receives the message from B then C can decrypt it using its own Private key. No other recipient other than C can decrypt the message because only C know C's private key.



Public Key Encryption

Components of Public Key Encryption

- **Plain Text:** This is the message which is readable or understandable. This message is given to the Encryption algorithm as an input.
- **Cipher Text:** The cipher text is produced as an output of Encryption algorithm. We cannot simply understand this message.
- **Encryption Algorithm:** The encryption algorithm is used to convert plain text into cipher text.
- **Decryption Algorithm:** It accepts the cipher text as input and the matching key (Private Key or Public key) and produces the original plain text
- **Public and Private Key:** One key either Private key (Secret key) or Public Key (known to everyone) is used for encryption and other is used for decryption

Public Key Infrastructure

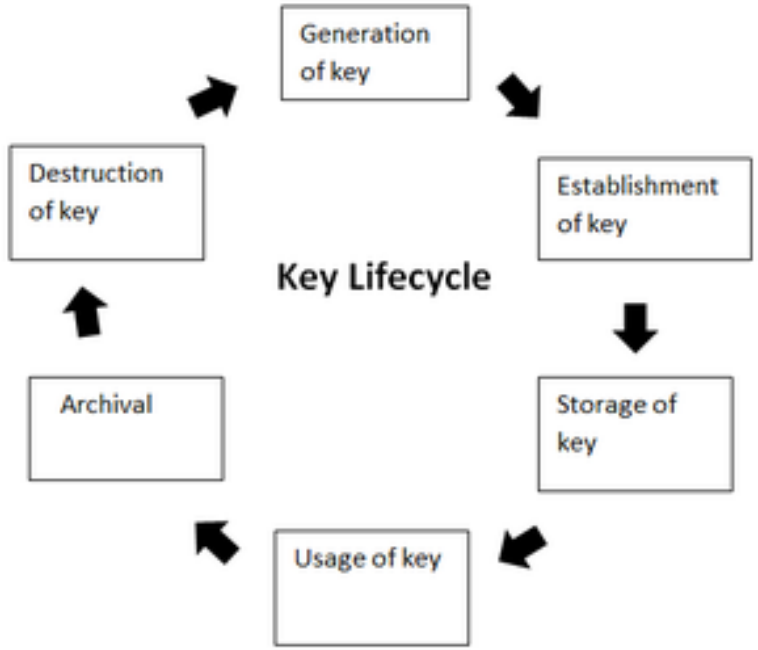
Public key infrastructure or PKI is the governing body behind issuing digital certificates. It helps to protect confidential data and gives unique identities to users and systems. Thus, it ensures security in communications.

The public key infrastructure uses a pair of keys: the public key and the private key to achieve security. The public keys are prone to attacks and thus an intact infrastructure is needed to maintain them.

Managing Keys in the Cryptosystem:

The security of a cryptosystem relies on its keys. Thus, it is important that we have a solid key management system in place. The 3 main areas of key management are as follows:

- A cryptographic key is a piece of data that must be managed by secure administration.
- It involves managing the key life cycle which is as follows:



- Public key management further requires:
    - **Keeping the private key secret:** Only the owner of a private key is authorized to use a private key. It should thus remain out of reach of any other person.
    - **Assuring the public key:** Public keys are in the open domain and can be publicly accessed. When this extent of public accessibility, it becomes hard to know if a key is correct and what it will be used for. The purpose of a public key must be explicitly defined.

Storage Security Evolution

When the first edition of this book was published almost ten years ago, 3.5-inch floppy disk drives were still included on some computers. Being portable storage devices, floppy disks were hard to secure. They were easily lost, or the data on them became corrupted. They could be used to propagate malware, either through files on the disk or through active code like the "girlfriend exploit" (as described in Chapter 2, named for the infamous practice of breaking into a network by giving a disk containing exploit software to a significant other who works there, and instructing her to run the program). The use of floppy disks was largely phased out by the late 2000s.

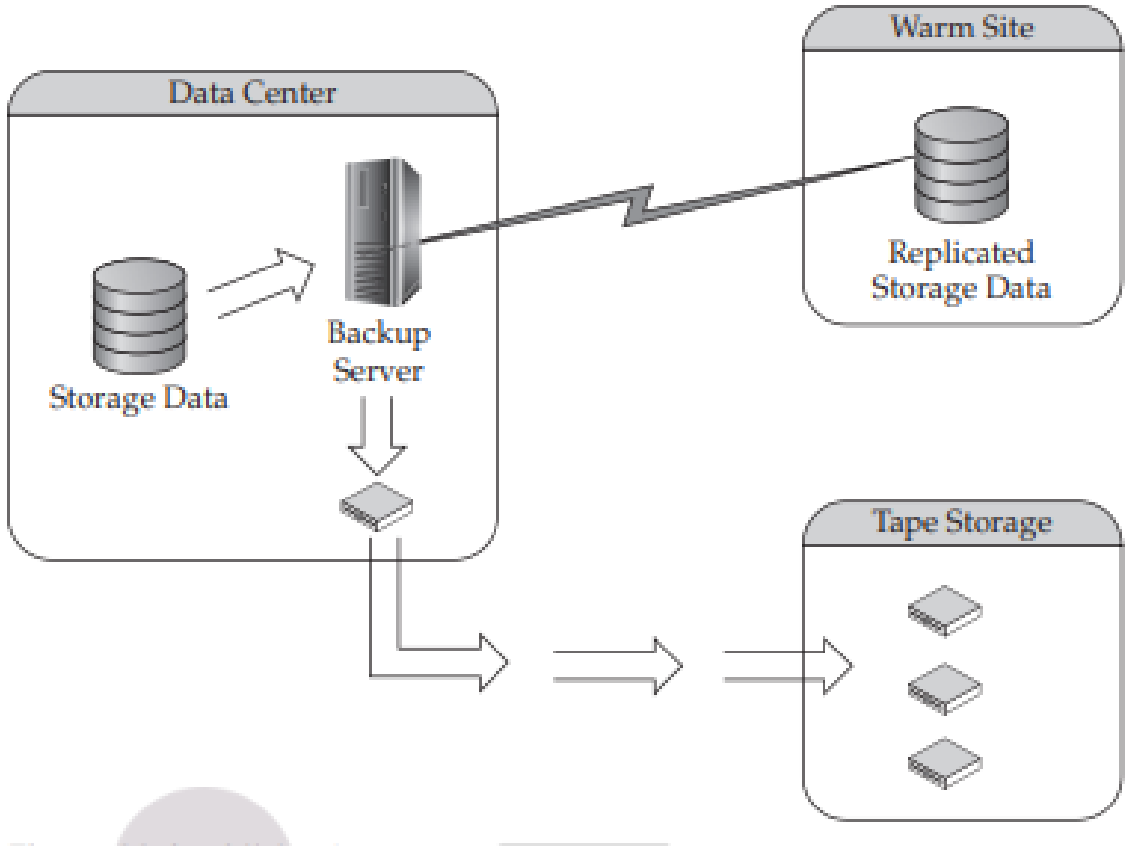
The next generation of storage devices, compact discs (CDs) and digital video discs (DVDs), posed a unique threat due to their longevity. Unlike other, more volatile storage media, these polycarbonate-encased metal optical data storage devices seem like they will last forever if handled properly. While optical discs are great for reliability and availability of data, their longevity elicits concerns of its own. If you place private, confidential data on a CD or DVD and then misplace the disc, who knows how long it might stick around and who may discover it in the future. For this reason, optical storage devices were banned in many corporate environments, especially those required to comply with privacy regulations. Moreover, once the data is burned to the media, it can't be changed, so you can't retroactively apply protection to it.

Flash drives (USB sticks and the like) have exploded in popularity over the past few years. These devices have become so cheap and prevalent that they have practically supplanted optical storage devices. Who needs to burn when you can simply copy? Nevertheless, while not as durable as optical storage, flash drives are similar to the archaic floppy disks of the past in many ways. They are omnipresent—many organizations try to ban their use, but everybody has one—so policies prohibiting these devices are hard to enforce outside of controlling the USB ports on every computer in the environment. They are prone to both malware and girlfriend exploits, in the same way floppies were—even more so, in the age of "autorun" (automatic execution of any code that is on the device, immediately upon connecting it). Flash drives are a significant source of malware infections in many environments. In addition, they make data theft remarkably easy with their small size, portability, and compatibility with every major computing platform.

Modern Storage Security

Modern storage solutions have moved away from the endpoint computers to the network. Network-attached storage (NAS) and storage area networks (SANs) consist of large hard drive arrays with a controller that serves up their contents on the network. NAS can be accessed by most computers and other devices on the network, while a SAN is typically used by servers.

These storage systems have many built-in security capabilities to choose from. Based on the security requirements of the environment, these security settings can be configured to meet the objectives of the security policy. Today's storage environments are complex. In fact, modern storage environments can be considered as separate IT infrastructures of their own. Many organizations are now dividing their IT organizations along the lines of networks, servers, and storage—acknowledging that storage merits a place alongside these long-venerated institutions.


Warm Site
Data Center
Replicated
Storage Data
Backup
Server
Storage Data
Tape Storage

Risk Remediation

This section categorizes the risks associated with data storage according to the classic CIA triad of Confidentiality, Integrity, and Availability (introduced in Chapter 4). For each identified risk, where possible, security controls consistent with the "three *D*s" of security (introduced in Chapter 1)—defense, detection, and deterrence—are applied in an effort to mitigate the risk using the principle of layered security (also known as defense-in-depth). What's left after those controls are applied to mitigate the risks is then identified as residual risks.

Confidentiality Risks

Confidentiality risks are associated with vulnerabilities and threats pertaining to the privacy and control of information, given that we want to make the information available in a controlled fashion to those who need it, without exposing it to unauthorized parties.

Data Leakage, Theft, Exposure, Forwarding

Data leakage is the risk of loss of information, such as confidential data and intellectual property, through intentional or unintentional means. There are four major threat vectors for data leakage: theft by outsiders, malicious sabotage by insiders (including unauthorized data printing, copying, or forwarding), inadvertent misuse by authorized users, and mistakes created by unclear policies.

- **Defense** Employ software controls to block inappropriate data access using a data loss prevention (DLP) solution and/or an information rights management (IRM) solution, as described in Chapters 8 and 9.
- **Detection** Use watermarking and data classification labeling along with monitoring software to track data flow.
- **Deterrence** Establish security policies that assign serious consequences to employees who leak data, and include clear language in contracts with service providers specifying how data privacy is to be protected and maintained, and what the penalties are for failure to protect and maintain it.
- **Residual risks** Data persistence within the storage environment can expose data long after it is no longer needed, especially if the storage is hosted on a vendor-provided service that dynamically moves data around in an untraceable manner. Administrative access that allows system administrators full access to all files, folders, and directories, as well as the underlying storage infrastructure itself, can expose private data to administrators.

Database Security

Most modern organizations rely heavily on the information stored in their database systems. From sales transactions to human resources records, mission-critical, sensitive data is tracked within these systems. From the standpoint of security, it is very important that business and systems administrators take the proper precautions to ensure that these systems and applications are as secure as possible. You wouldn't want a junior-level database administrator to be able to access information that only the executive team should see; but you also wouldn't want to prevent your staff from doing their jobs. As with all security implementations, the key is to find a balance between security and usability. All of the security-related best practices that have been laid out throughout this book apply to securing databases. These include network-level security, physical security, and using server-related best practices. However, there are additional considerations that should be taken into account when securing databases. In this chapter, we'll look at some of these special concepts and techniques. Specifically, we'll begin by taking a look at some general information about what makes databases special. Then, we'll look at the various levels of permissions that must be implemented and managed before a database can be considered secure. We'll also look at database auditing.

General Database Security Concepts

Modern databases must meet different goals. They must be reliable, provide for quick access to information, and provide advanced features for data storage and analysis. Furthermore, they must be flexible enough to adapt to many different scenarios and types of usage. Many organizations rely on databases to serve as the "back end" for purchased applications or custom-developed applications. The "front end" of these systems are generally client applications or web user interfaces.

Architecturally, relational databases function in a client-server manner (although they can certainly be used as part of multitier applications). That is, a client computer, application, or user can only communicate directly with the database services that are running. They cannot directly access the database files, as can be done with "desktop" database systems, such as Microsoft Access. This is an important point, since it allows security configuration and management to occur at the database level, instead of leaving that responsibility to users and applications.

Databases can be used in various capacities, including:

- **Application support** Ranging from simple employee lists to enterprise-level tracking software, relational databases are the most commonly used method for storing data. Through the use of modern databases, users and developers can rely on security, scalability, and recoverability features.
- **Secure storage of sensitive information** Relational databases offer one of the most secure methods of centrally storing important data. As we'll see throughout this chapter, there are many ways in which access to data can be defined and enforced. These methods can be used to meet legislative requirements in regulated industries (for example, the HIPAA standard for storing and transferring healthcare-related information) and generally for storing important data.
- **Online transaction processing (OLTP)** OLTP services are often the most common functions of databases in many organizations. These systems are responsible for receiving and storing information that is accessed by client applications and other servers. OLTP databases are characterized by having a high level of data modification (inserting, updating, and deleting rows). Therefore, they are optimized to support dynamically changing data. Generally, they store large volumes of information that can balloon very quickly if not managed properly.
- **Data warehousing** Many organizations go to great lengths to collect and store as much information as possible. But what good is this information if it can't easily be analyzed? The primary business reason for storing many types of information is to use this data eventually to help make business decisions. Although reports can be generated against OLTP databases, there are several potential problems: Reports might take a long time to run, and thus tax system resources. If reports are run against a production OLTP server, overall system performance can be significantly decreased. OLTP servers are not optimized for the types of queries used in reporting, thus making the problem worse. Reporting requirements are very different. In reporting systems, the main type of activity is data analysis. OLTP systems get bogged down when the amount of data in the databases gets very large. Therefore, production OLTP data must be often archived to other media or stored in another data repository. Relational database platforms can serve as a repository for information collected from many different data sources within an organization. This database can then be used for centralized reporting and by "decision support" systems.